Importing The Modules

In [79]:
```
import pandas as pd
import numpy as np
```

1.Load Data

In [80]:
```
df = pd.read_csv('dc_motor_ml_ready_dataset.csv')
df.head() # Display the first few rows of the dataset
```

Out[80]:

	INPUT_VOLTAGE	TORQUE	SPEED_0	SPEED_1	Armature Resistance	Armature Inductance	Torque Constant	C
0	39.392241	0.284950	274.747314	274.747314	1.806450	0.009658	0.179281	0
1	14.553942	0.089908	102.885150	102.885150	1.804283	0.009763	0.178769	0
2	45.796941	0.127876	335.471653	335.471653	1.806202	0.009762	0.178786	0
3	40.640684	0.012803	303.896011	303.896011	1.803110	0.009748	0.179104	0
4	18.774211	0.250822	121.728152	121.728152	1.806195	0.009747	0.179170	0

In [81]:
```
df.info() # Get a summary of the dataset
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 10 columns):
 #   Column                Non-Null Count  Dtype  
---  ------                --------------  -----  
 0   INPUT_VOLTAGE         5000 non-null   float64
 1   TORQUE                5000 non-null   float64
 2   SPEED_0               5000 non-null   float64
 3   SPEED_1               5000 non-null   float64
 4   Armature Resistance   5000 non-null   float64
 5   Armature Inductance   5000 non-null   float64
 6   Torque Constant       5000 non-null   float64
 7   Back EMF Constant     5000 non-null   float64
 8   Friction Coefficient  5000 non-null   float64
 9   Moment of Inertia     5000 non-null   float64
dtypes: float64(10)
memory usage: 390.8 KB
```

In [82]:
```
expected_columns = [
"INPUT_VOLTAGE",
"TORQUE",
"SPEED_0",
"SPEED_1",
"Armature Resistance",
"Armature Inductance ",
```

```
"Torque Constant",
"Back EMF Constant",
"Friction Coefficient",
"Moment of Inertia"
 ]
missing_col = [col for col in expected_columns if col not in
df.columns]
print("Missing columns: ", len(missing_col))
```

```
Missing columns:  0
```

In [83]:
```
df.isnull().sum() # Check for missing values in each column
```

Out[83]:
```
INPUT_VOLTAGE           0
TORQUE                  0
SPEED_0                 0
SPEED_1                 0
Armature Resistance     0
Armature Inductance     0
Torque Constant         0
Back EMF Constant       0
Friction Coefficient    0
Moment of Inertia       0
dtype: int64
```

In [84]:
```
df.fillna(df.mean(numeric_only=True)) # Fill missing values with column means
```

Out[84]:

	INPUT_VOLTAGE	TORQUE	SPEED_0	SPEED_1	Armature Resistance	Armature Inductance	Torque Constant
0	39.392241	0.284950	274.747314	274.747314	1.806450	0.009658	0.179281
1	14.553942	0.089908	102.885150	102.885150	1.804283	0.009763	0.178769
2	45.796941	0.127876	335.471653	335.471653	1.806202	0.009762	0.178786
3	40.640684	0.012803	303.896011	303.896011	1.803110	0.009748	0.179104
4	18.774211	0.250822	121.728152	121.728152	1.806195	0.009747	0.179170
...	...	...	...	...	...	...	...
4995	35.715137	0.227476	163.727694	163.727694	1.990931	0.009833	0.200000
4996	25.300542	0.261403	111.634929	111.634929	1.992263	0.009812	0.199911
4997	29.199496	0.014518	143.121182	143.121182	1.989486	0.009858	0.200000
4998	37.106694	0.064558	179.716080	179.716080	1.990868	0.009833	0.200000
4999	40.830664	0.253266	189.167717	189.167717	1.988549	0.009786	0.200000

5000 rows × 10 columns

In [85]:
```
df.describe() # Get statistical summary of the dataset
```

Out[85]:

	INPUT_VOLTAGE	TORQUE	SPEED_0	SPEED_1	Armature Resistance	Armature Inductance
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000
mean	29.676615	0.201195	146.990517	146.990517	1.967481	0.009896
std	10.391407	0.115635	59.392410	59.392410	0.048755	0.000120
min	12.003906	0.000062	39.238075	39.238075	1.803110	0.009332
25%	20.641709	0.102003	96.785739	96.785739	1.968641	0.009844
50%	29.569446	0.202170	144.231901	144.231901	1.990731	0.009937
75%	38.561938	0.301901	191.179852	191.179852	1.996891	0.009994
max	47.986646	0.399969	348.514544	348.514544	2.000000	0.010000

2.Feature Selection

In [86]:
```
input_features = [
 "INPUT_VOLTAGE",
 "TORQUE",
 "SPEED_0",
 "SPEED_1"
]
target_features = [
 "Armature Resistance",
 "Armature Inductance ",
 "Torque Constant",
 "Back EMF Constant",
 "Friction Coefficient",
 "Moment of Inertia"
]
required_features = input_features + target_features
X = df[input_features]
y = df[target_features]
required = df[required_features]
print("Input features shape: ", X.shape)
print("Target features shape: ", y.shape)
print("Required features shape: ", required.shape)
required.to_csv('motor_DC_required_features.csv', index=False)
# Save the required features to a new CSV file
```

```
Input features shape:  (5000, 4)
Target features shape:  (5000, 6)
Required features shape:  (5000, 10)
```

In [87]:
```
import os
import matplotlib.pyplot as plt
os.makedirs("distrubutions",exist_ok=True)
for col in required_features:
 plt.figure(figsize=(8,6))
 df[col].hist(bins=30,color='skyblue',edgecolor='black')
```

```
plt.title(f'Distribution of {col}')
plt.xlabel(col)
plt.ylabel('Frequency')
plt.grid(True)
plot_path = f"distrubutions/{col}_histogram.png"
plt.savefig(plot_path)
plt.close()
print(f"Histogram for {col} saved to {plot_path}")
```

```
Histogram for INPUT_VOLTAGE saved to distrubutions/INPUT_VOLTAGE_histogram.png
Histogram for TORQUE saved to distrubutions/TORQUE_histogram.png
Histogram for SPEED_0 saved to distrubutions/SPEED_0_histogram.png
Histogram for SPEED_1 saved to distrubutions/SPEED_1_histogram.png
Histogram for Armature Resistance saved to distrubutions/Armature Resistance_histogr
am.png
Histogram for Armature Inductance  saved to distrubutions/Armature Inductance _histo
gram.png
Histogram for Torque Constant saved to distrubutions/Torque Constant_histogram.png
Histogram for Back EMF Constant saved to distrubutions/Back EMF Constant_histogram.p
ng
Histogram for Friction Coefficient saved to distrubutions/Friction Coefficient_histo
gram.png
Histogram for Moment of Inertia saved to distrubutions/Moment of Inertia_histogram.p
ng
```

3.Train-Test Split

In [88]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,      # 20% test data
    random_state=42     # for reproducibility
)
print("Training samples:", X_train.shape)
print("Testing samples:", X_test.shape)
```

```
Training samples: (4000, 4)
Testing samples: (1000, 4)
```

4.Model Selection

Linear Regression

In [89]:
```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv("dc_motor_ml_ready_dataset.csv")   # dataset

input_features = [
 "INPUT_VOLTAGE",
 "TORQUE",
 "SPEED_0",
 "SPEED_1"
]
target_features = [
 "Armature Resistance",
 "Armature Inductance ",
 "Torque Constant",
 "Back EMF Constant",
 "Friction Coefficient",
 "Moment of Inertia"
]
required_features = input_features + target_features
X = df[input_features]
y = df[target_features]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("R² Score:", r2)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)
```

```
MSE: 9.831541727891907e-05
R² Score: 0.41533376340124367
Intercept: [1.97003018e+00 9.89594184e-03 1.96981228e-01 1.86355906e-01
 4.81040680e-04 4.81524654e-04]
Coefficients: [[ 1.15100388e-02 -1.13771086e-01 -1.09260507e-03 -1.09260507e-03]
 [-1.82661392e-07  2.53145848e-05  1.01173541e-09  1.01173541e-09]
 [ 3.46991657e-04 -2.79897068e-03 -3.30258049e-05 -3.30258049e-05]
 [ 4.82049116e-03 -4.67944525e-02 -4.55486123e-04 -4.55486123e-04]
 [ 5.64996626e-06 -4.60635401e-05 -5.40252810e-07 -5.40252810e-07]
 [ 8.43330033e-06 -7.53104519e-05 -8.06714318e-07 -8.06714318e-07]]
```

Ridge Regression

In [90]:
```python
from sklearn.linear_model import RidgeCV

alphas = np.logspace(-3, 3, 20)

model = RidgeCV(alphas=alphas, cv=5)
model.fit(X_train, y_train)

print("Best alpha:", model.alpha_)

y_pred = model.predict(X_test)
```

```
Best alpha: 0.0379269019073225
```

In [91]:
```python
from sklearn.linear_model import Ridge

model = Ridge(alpha=0.0379269019073225)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

from sklearn.ensemble import RandomForestRegressor

model = RandomForestRegressor(
    n_estimators=200,
    max_depth=12,
    random_state=42
)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("R² Score:", r2)
```

```
MSE: 4.8021381656485224e-05
R² Score: 0.5162112996573485
```

Random Forest Regressor

In [92]:
```python
import pandas as pd
import numpy as np
```

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score
split = int(0.8 * len(df))

X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

base_model = RandomForestRegressor(
    n_estimators=500,
    max_depth=15,
    min_samples_leaf=5,
    random_state=42
)

model = MultiOutputRegressor(base_model)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

targets = y.columns

for i, name in enumerate(targets):
    mse = mean_squared_error(y_test.iloc[:, i], y_pred[:, i])
    r2 = r2_score(y_test.iloc[:, i], y_pred[:, i])

    print(f"{name}:  MSE = {mse:.4f},  R² = {r2:.4f}")
```

```
Armature Resistance:  MSE = 0.0001,  R² = -1.1921
Armature Inductance :  MSE = 0.0000,  R² = -0.0191
Torque Constant:  MSE = 0.0000,  R² = -0.2145
Back EMF Constant:  MSE = 0.0000,  R² = 0.6306
Friction Coefficient:  MSE = 0.0000,  R² = -0.1504
Moment of Inertia:  MSE = 0.0000,  R² = -0.0432
```

Using Rmse as R_square and MSE do not work much here

In [93]:
```
from sklearn.preprocessing import StandardScaler

y_scaler = StandardScaler()
y_scaled = y_scaler.fit_transform(y)

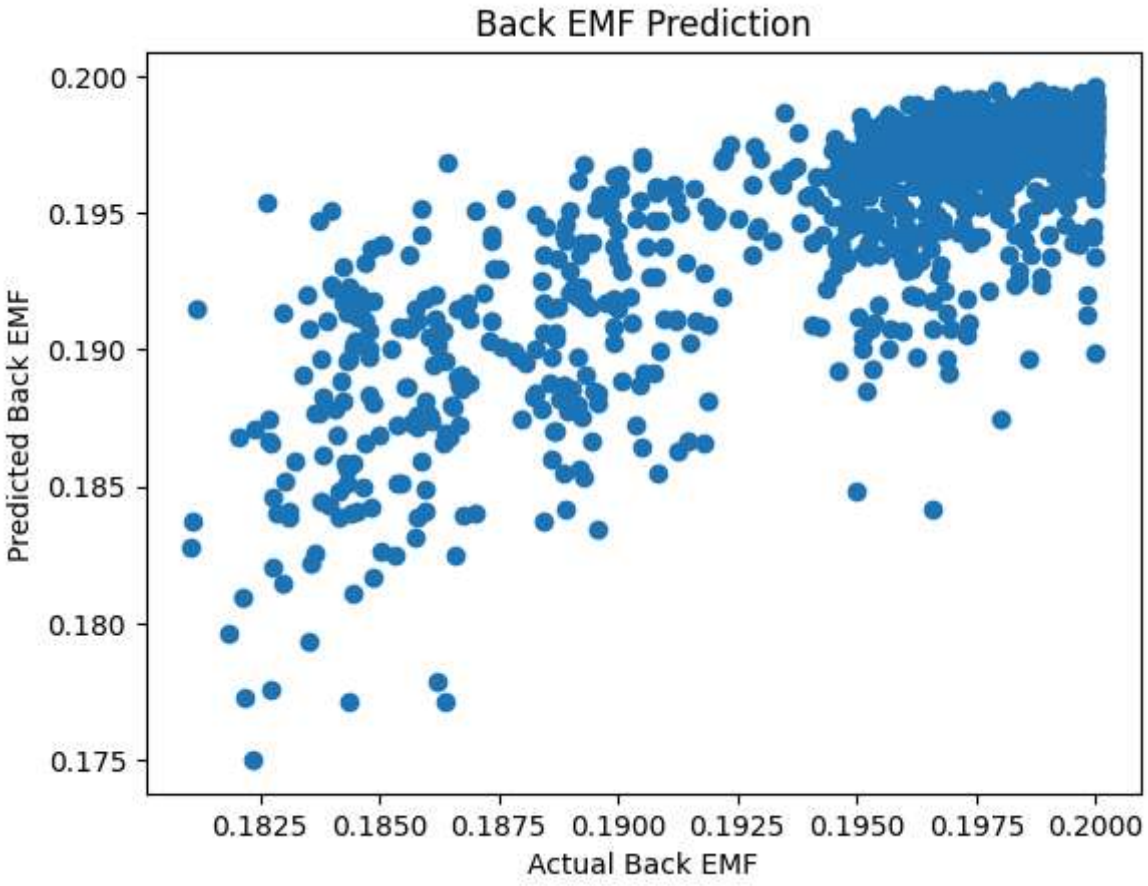
from sklearn.metrics import mean_squared_error
import numpy as np

for i, name in enumerate(y.columns):
    mse = mean_squared_error(y_test.iloc[:, i], y_pred[:, i])
    rmse = np.sqrt(mse)
    print(f"{name}: RMSE = {rmse:.6f}")
```

```
Armature Resistance: RMSE = 0.010530
Armature Inductance : RMSE = 0.000108
Torque Constant: RMSE = 0.002045
Back EMF Constant: RMSE = 0.003064
Friction Coefficient: RMSE = 0.000011
Moment of Inertia: RMSE = 0.000012
```

In [94]:
```
import matplotlib.pyplot as plt

plt.scatter(y_test['Back EMF Constant'], y_pred[:, 3])
plt.xlabel("Actual Back EMF")
plt.ylabel("Predicted Back EMF")
plt.title("Back EMF Prediction")
plt.show()
```



Back emf is accurately identifiable(Adding more parameters to make other parameters strong)

In [ ]:
```
delta_t=0.49
df['Acceleration'] = (df['SPEED_0'] - df['SPEED_1']) / delta_t
df['Speed_Avg'] = (df['SPEED_0'] + df['SPEED_1']) / 2
df['Speed_Diff'] = df['SPEED_0'] - df['SPEED_1']
df['Speed_sq'] = df['Speed_Avg'] ** 2  # captures nonlinear friction effects
df['Estimated_Current'] = (df['INPUT_VOLTAGE'] - df['Speed_Avg']*df['Back EMF Const
df['Voltage_x_Acceleration'] = df['INPUT_VOLTAGE'] * df['Acceleration']
df['Torque_x_Speed'] = df['TORQUE'] * df['Speed_Avg']
```

Random Forest Regressor(having strong parameters)

In [96]:
```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.multioutput import MultiOutputRegressor
```

```
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np
import matplotlib.pyplot as plt

input_features = [
 "INPUT_VOLTAGE",
 "TORQUE",
 "SPEED_0",
 "SPEED_1",
 "Acceleration",
 "Speed_Avg",
 "Speed_Diff",
 "Speed_sq",
 "Estimated_Current",
 "Voltage_x_Acceleration",
 "Torque_x_Speed"
]
target_features = [
 "Armature Resistance",
 "Armature Inductance ",
 "Torque Constant",
 "Back EMF Constant",
 "Friction Coefficient",
 "Moment of Inertia"
]

X =df[input_features]
y =df[target_features]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

base_model = RandomForestRegressor(
    n_estimators=500,
    max_depth=15,
    min_samples_leaf=5,
    random_state=42
)

model = MultiOutputRegressor(base_model)
model.fit(X_train, y_train)

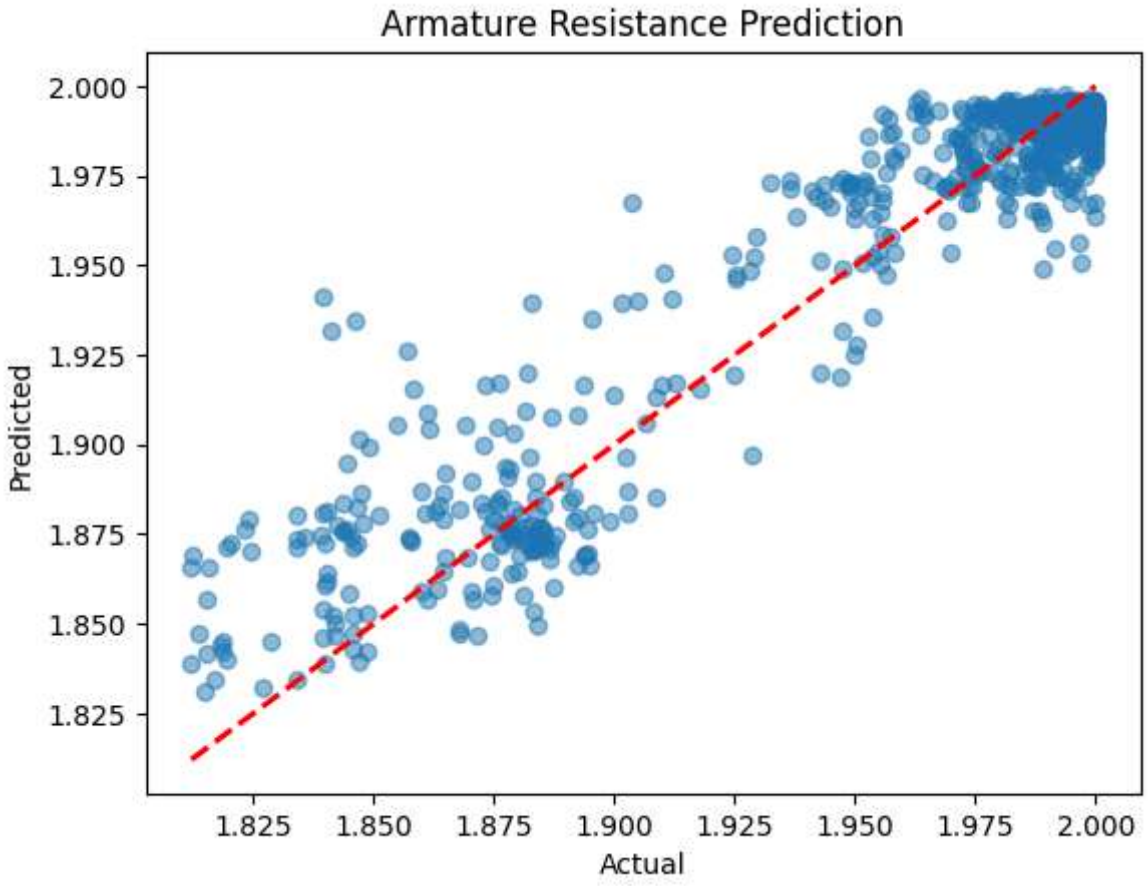
y_pred = model.predict(X_test)

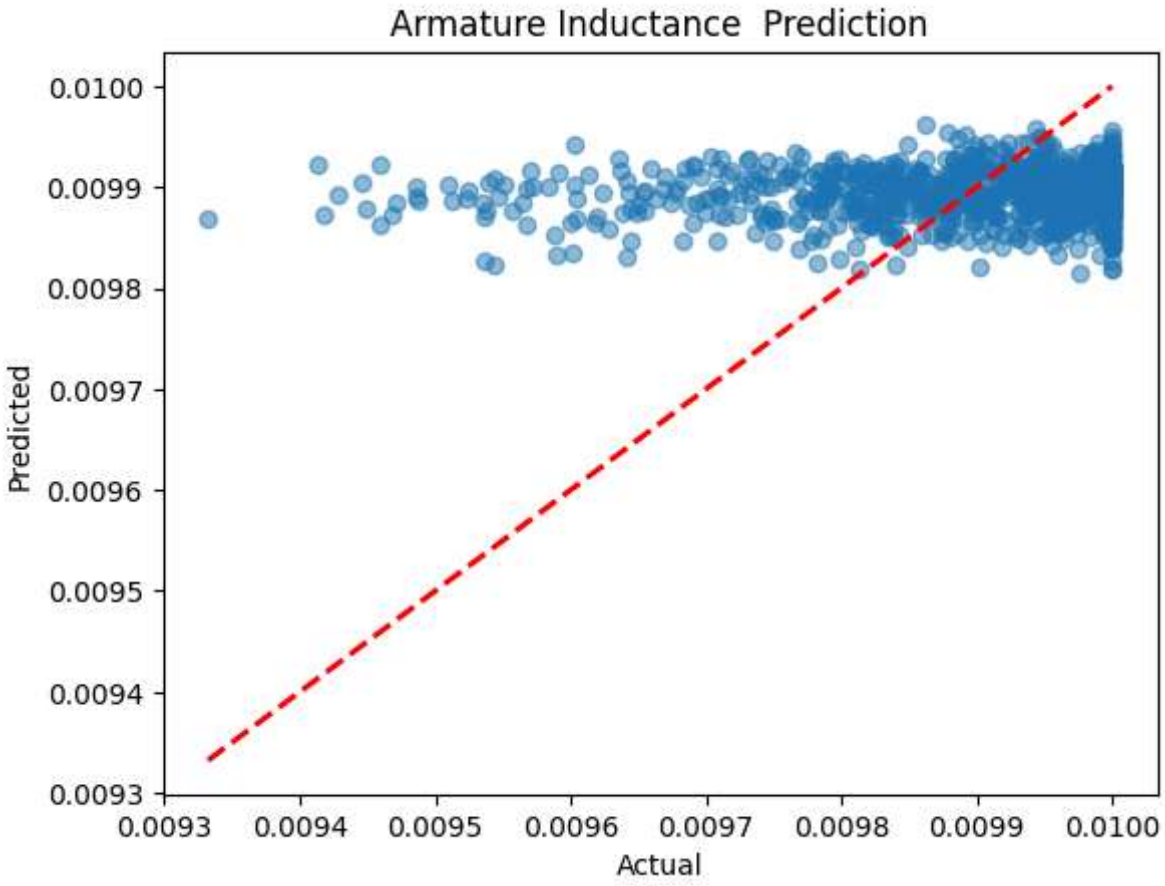
for i, col in enumerate(y.columns):
    rmse = np.sqrt(mean_squared_error(y_test.iloc[:, i], y_pred[:, i]))
    r2 = r2_score(y_test.iloc[:, i], y_pred[:, i])
    print(f"{col} --> RMSE: {rmse:.6f}, R²: {r2:.4f}")

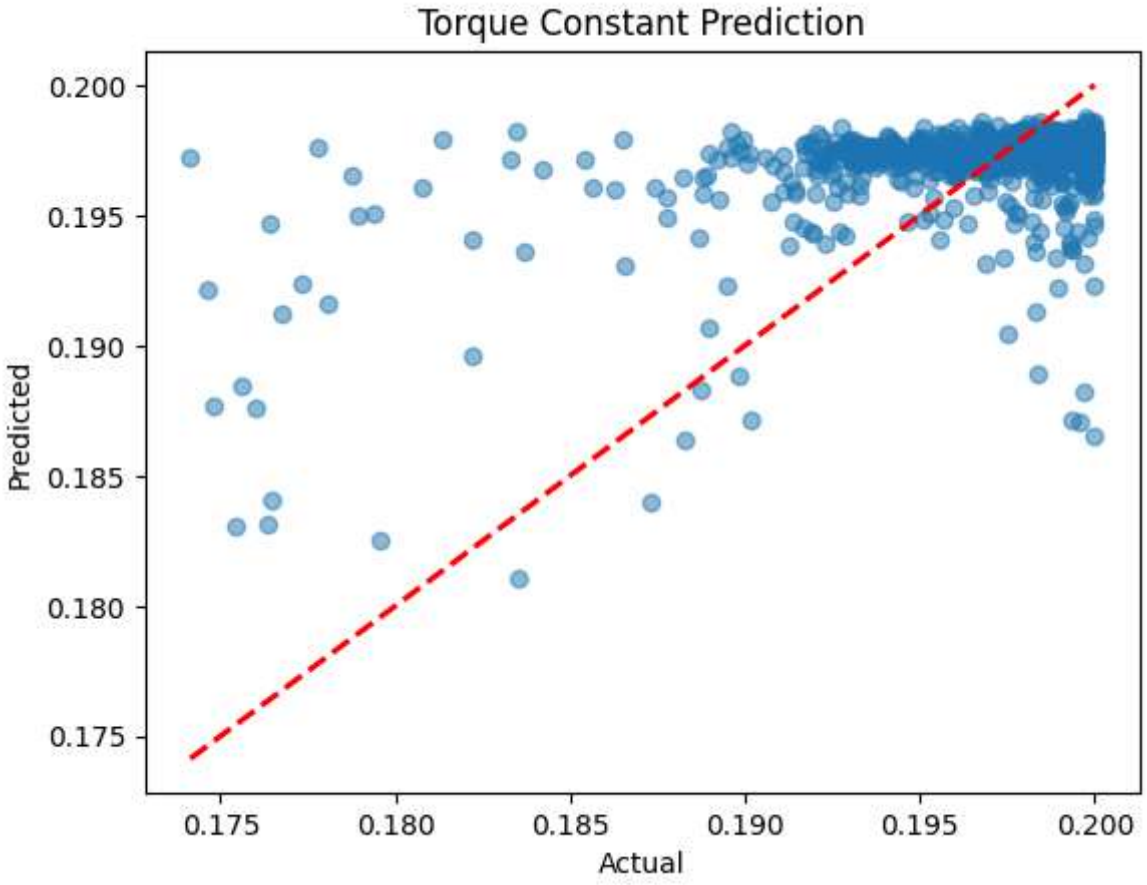
for i, col in enumerate(y.columns):
    plt.figure()
    plt.scatter(y_test.iloc[:, i], y_pred[:, i], alpha=0.5)
    plt.plot([y_test.iloc[:, i].min(), y_test.iloc[:, i].max()],
             [y_test.iloc[:, i].min(), y_test.iloc[:, i].max()],
             'r--', lw=2)
```

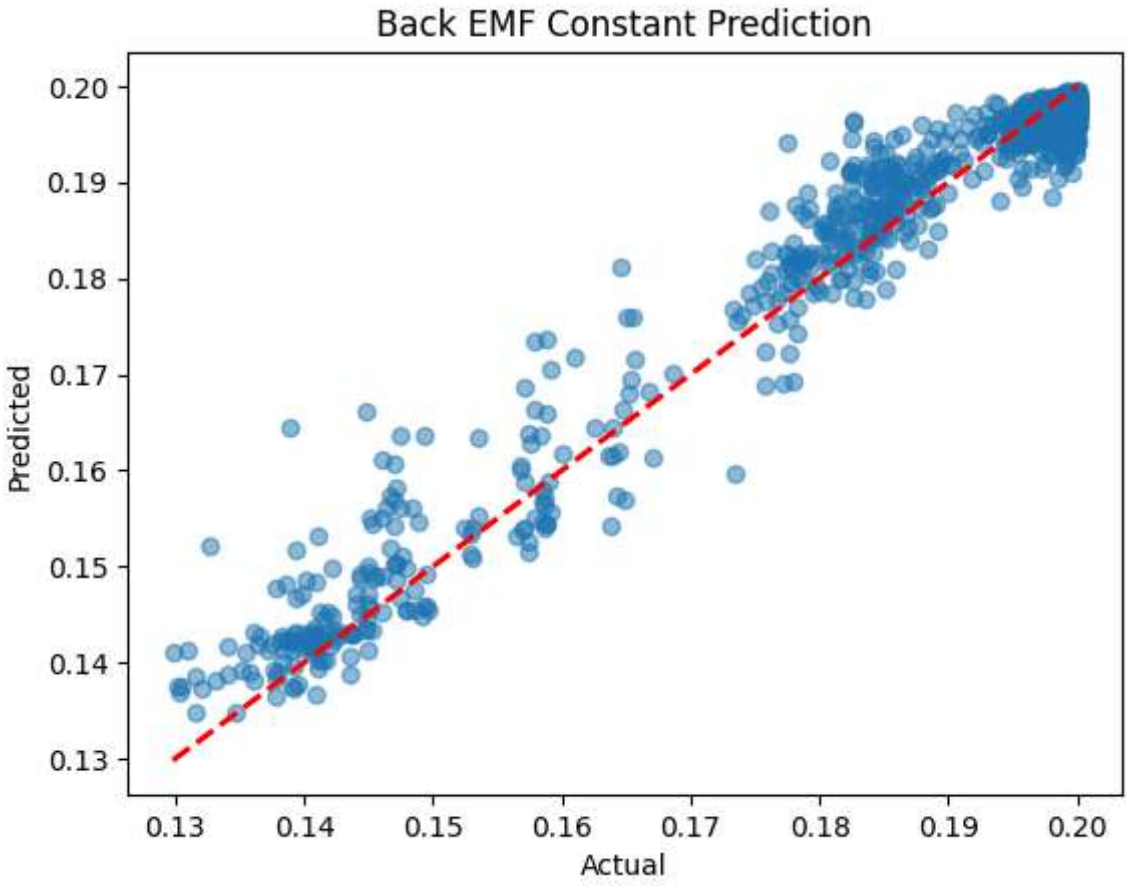
```
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.title(f"{col} Prediction")
    plt.show()
```

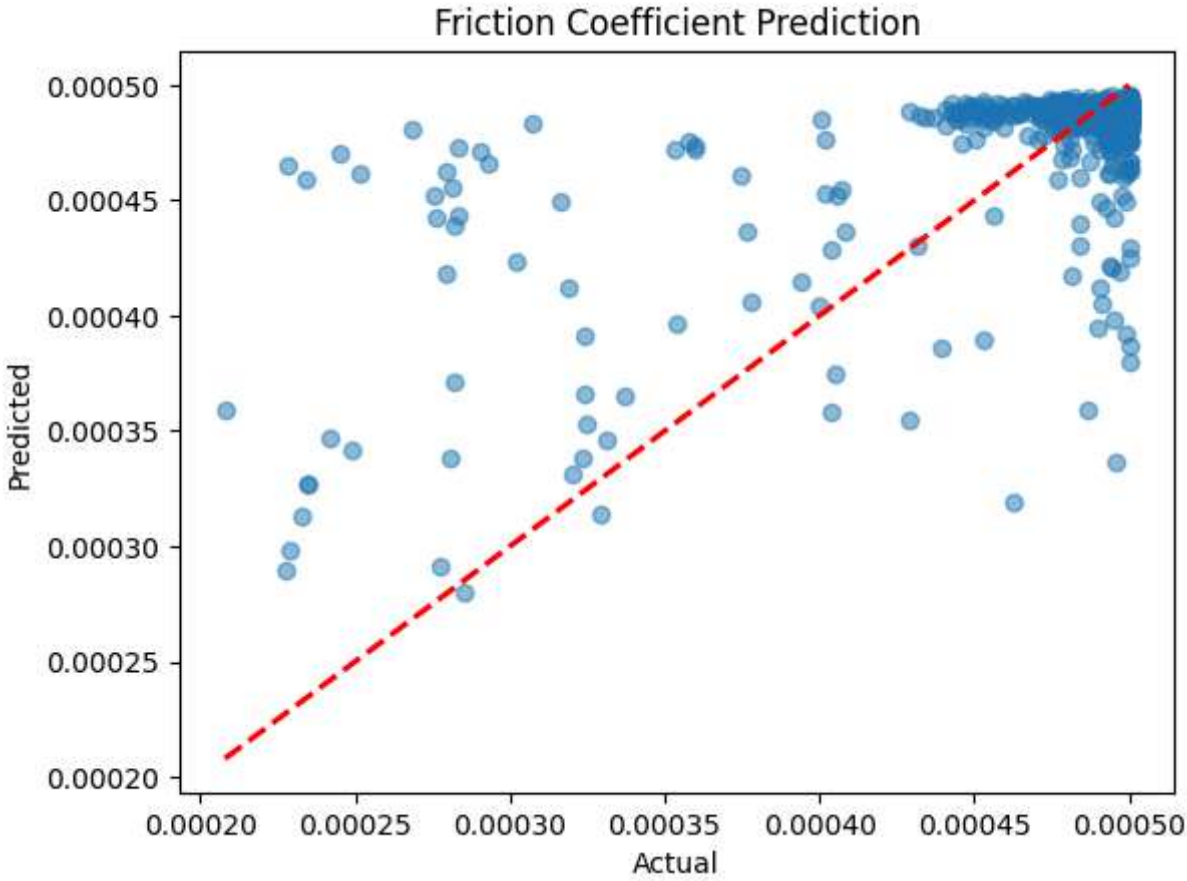
```
Armature Resistance --> RMSE: 0.015787, R²: 0.9045
Armature Inductance  --> RMSE: 0.000120, R²: -0.0053
Torque Constant --> RMSE: 0.003526, R²: 0.2304
Back EMF Constant --> RMSE: 0.004040, R²: 0.9603
Friction Coefficient --> RMSE: 0.000034, R²: 0.4274
Moment of Inertia --> RMSE: 0.000045, R²: 0.4810
```

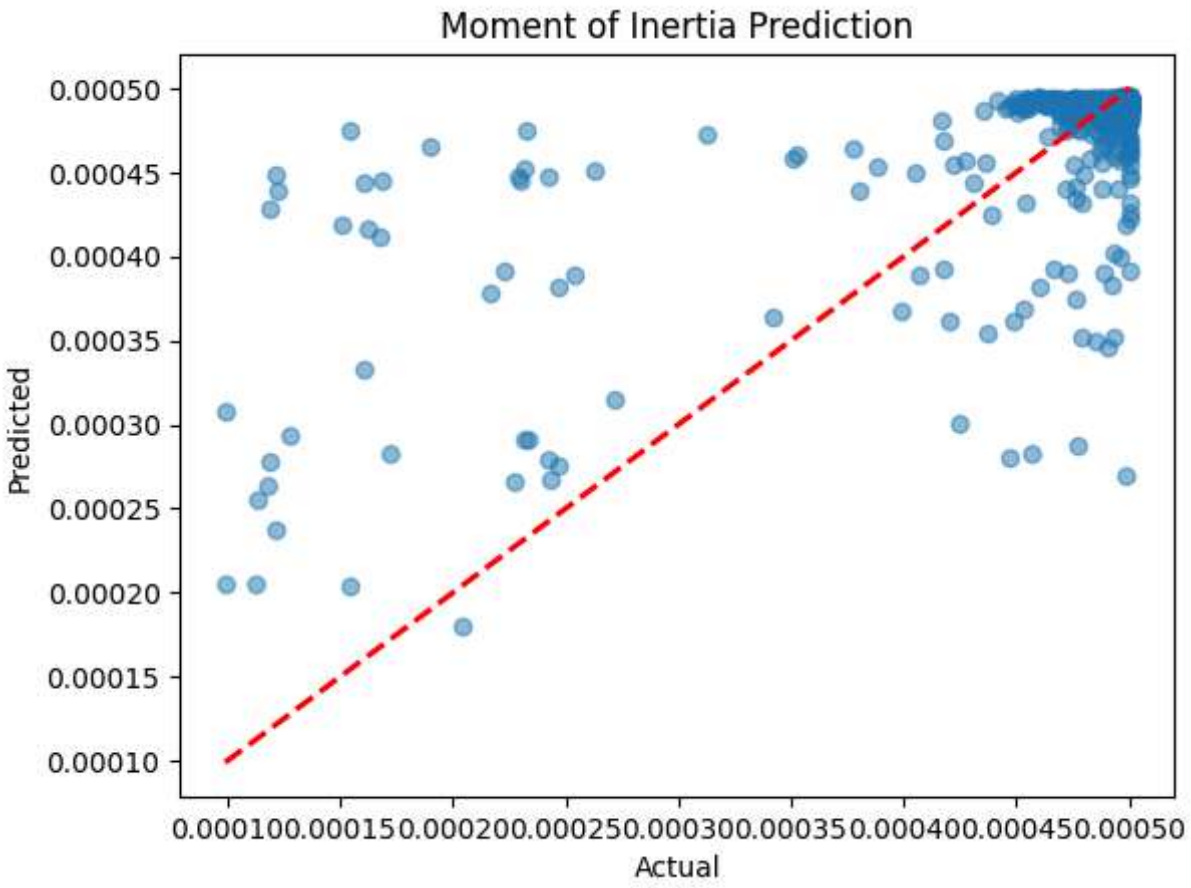



Armature Inductance Prediction
Predicted
Actual
0.0100
0.0099
0.0098
0.0097
0.0096
0.0095
0.0094
0.0093
0.0093
0.0094
0.0095
0.0096
0.0097
0.0098
0.0099
0.0100


Torque Constant Prediction
Predicted
Actual
0.200
0.195
0.190
0.185
0.180
0.175
0.175
0.180
0.185
0.190
0.195
0.200


Back EMF Constant Prediction
Predicted
Actual
0.13
0.14
0.15
0.16
0.17
0.18
0.19
0.20


Friction Coefficient Prediction
Predicted
Actual
0.00020
0.00025
0.00030
0.00035
0.00040
0.00045
0.00050



Gradient Boosting Regressor

In [97]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

input_features = [
 "INPUT_VOLTAGE",
 "TORQUE",
 "SPEED_0",
 "SPEED_1",
 "Acceleration",
 "Speed_Avg",
 "Speed_Diff",
 "Speed_sq",
 "Estimated_Current",
 "Voltage_x_Acceleration",
 "Torque_x_Speed"
]
target_features = [
 "Armature Resistance",
 "Armature Inductance ",
 "Torque Constant",
 "Back EMF Constant",
```

```
 "Friction Coefficient",
 "Moment of Inertia"
]

X = df[input_features]
y = df[target_features]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

base_model = GradientBoostingRegressor(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=5,
    min_samples_leaf=5,
    random_state=42
)

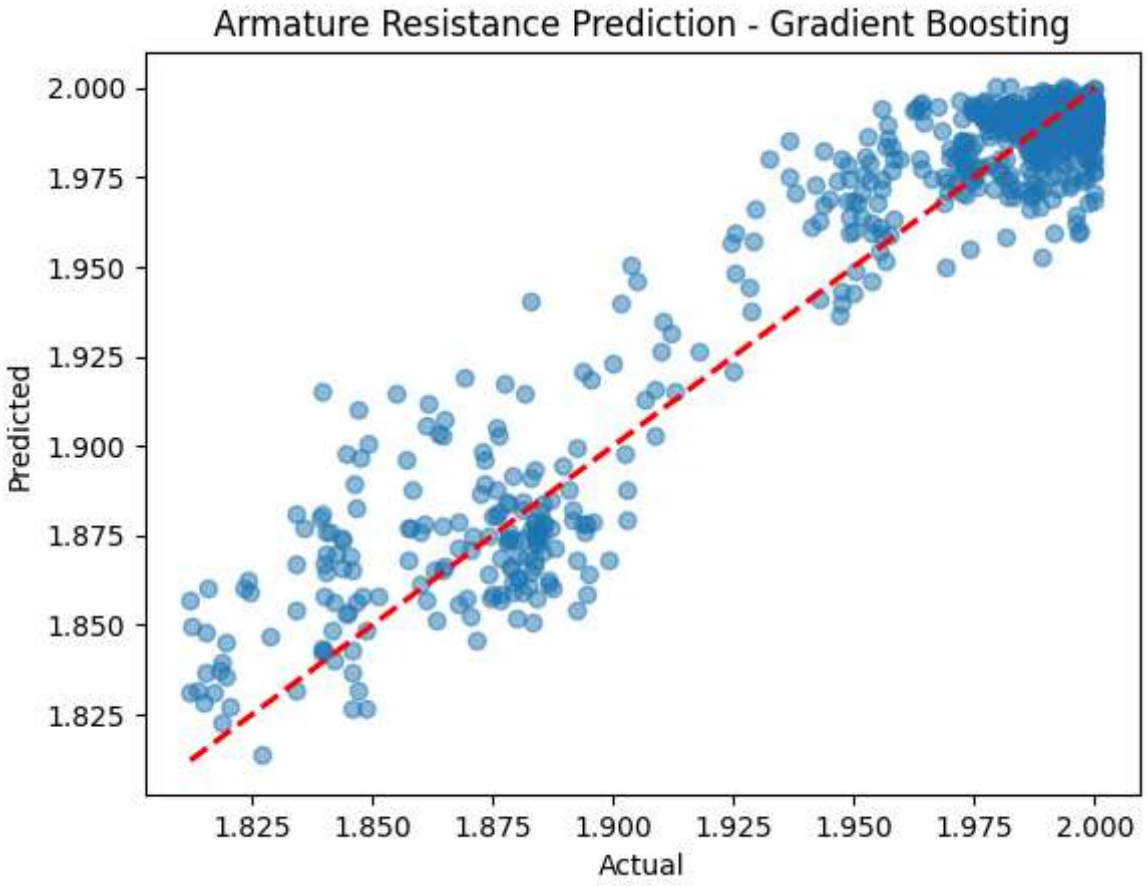
model = MultiOutputRegressor(base_model)
model.fit(X_train, y_train)

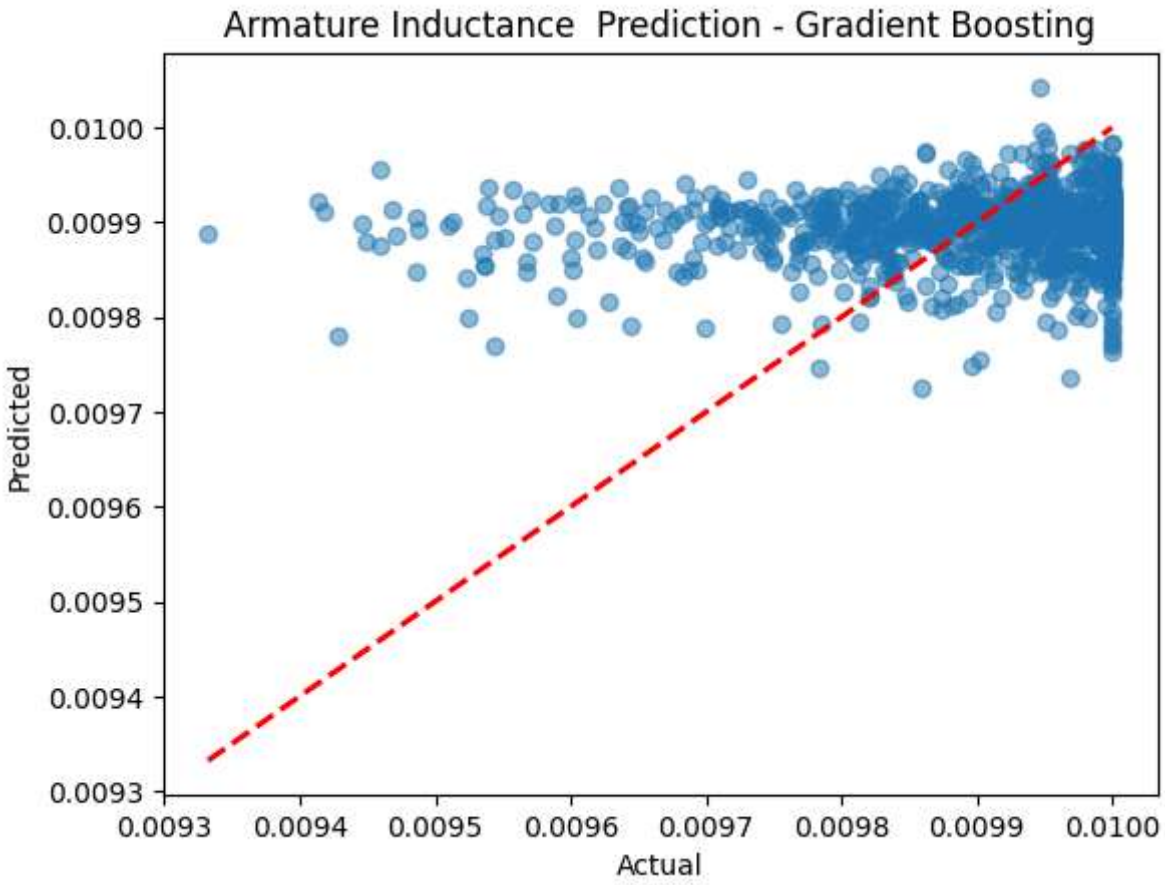
y_pred = model.predict(X_test)

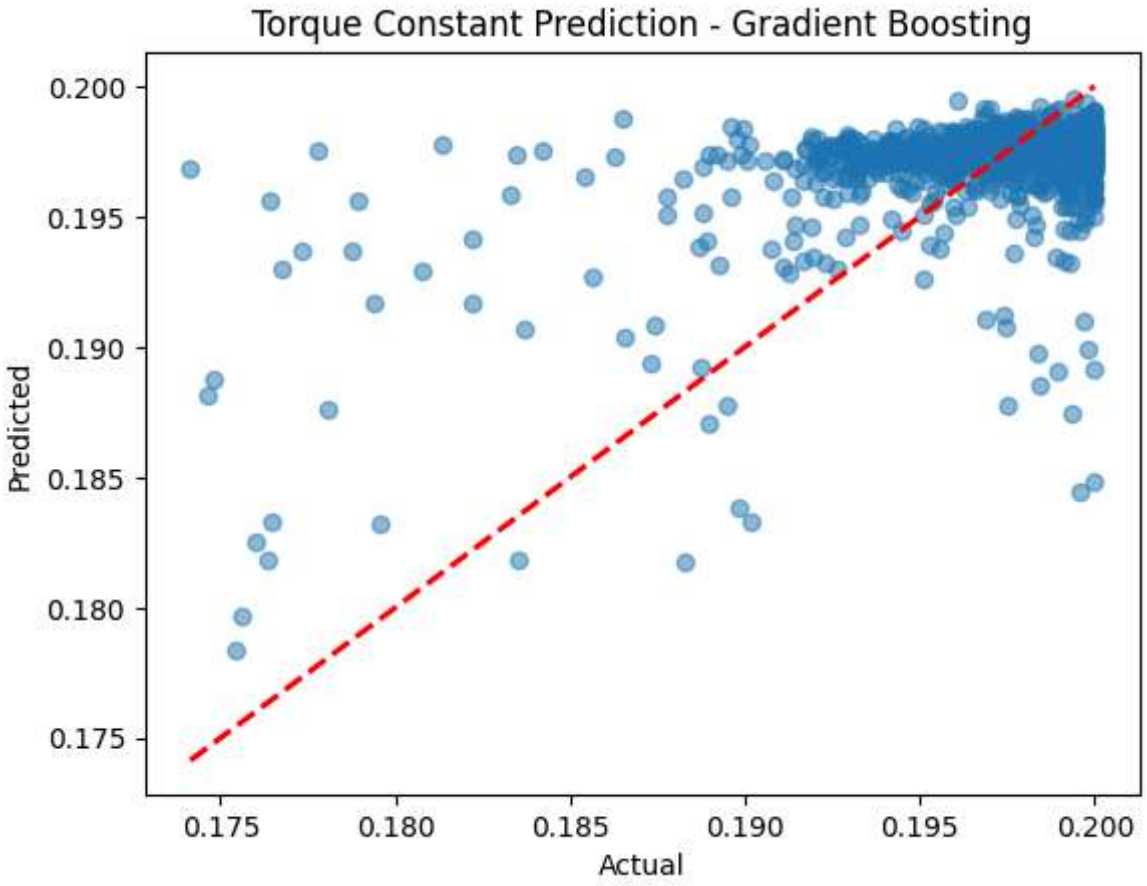
print("=== Gradient Boosting Multi-Output Results ===")
for i, col in enumerate(y.columns):
    rmse = np.sqrt(mean_squared_error(y_test.iloc[:, i], y_pred[:, i]))
    r2 = r2_score(y_test.iloc[:, i], y_pred[:, i])
    print(f"{col} --> RMSE: {rmse:.6f}, R²: {r2:.4f}")

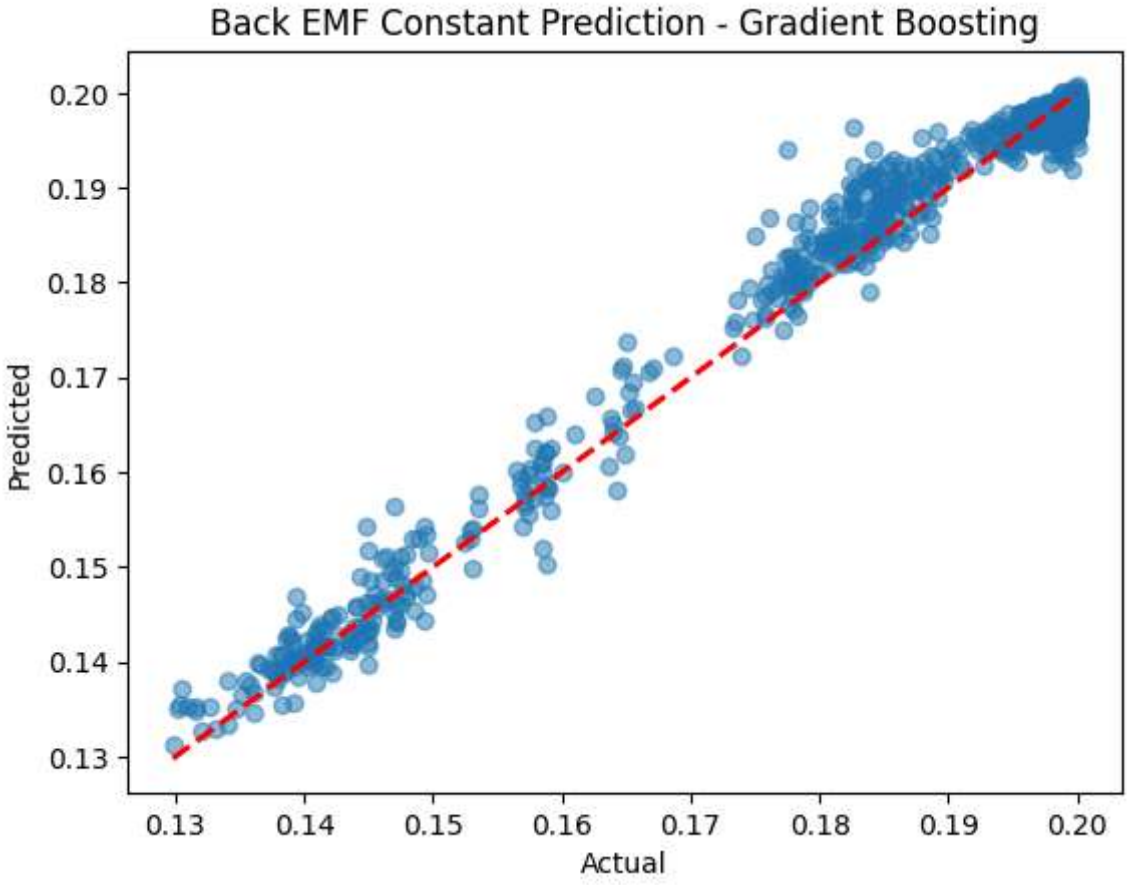
for i, col in enumerate(y.columns):
    plt.figure()
    plt.scatter(y_test.iloc[:, i], y_pred[:, i], alpha=0.5)
    plt.plot([y_test.iloc[:, i].min(), y_test.iloc[:, i].max()],
             [y_test.iloc[:, i].min(), y_test.iloc[:, i].max()],
             'r--', lw=2)
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.title(f"{col} Prediction - Gradient Boosting")
    plt.show()
```

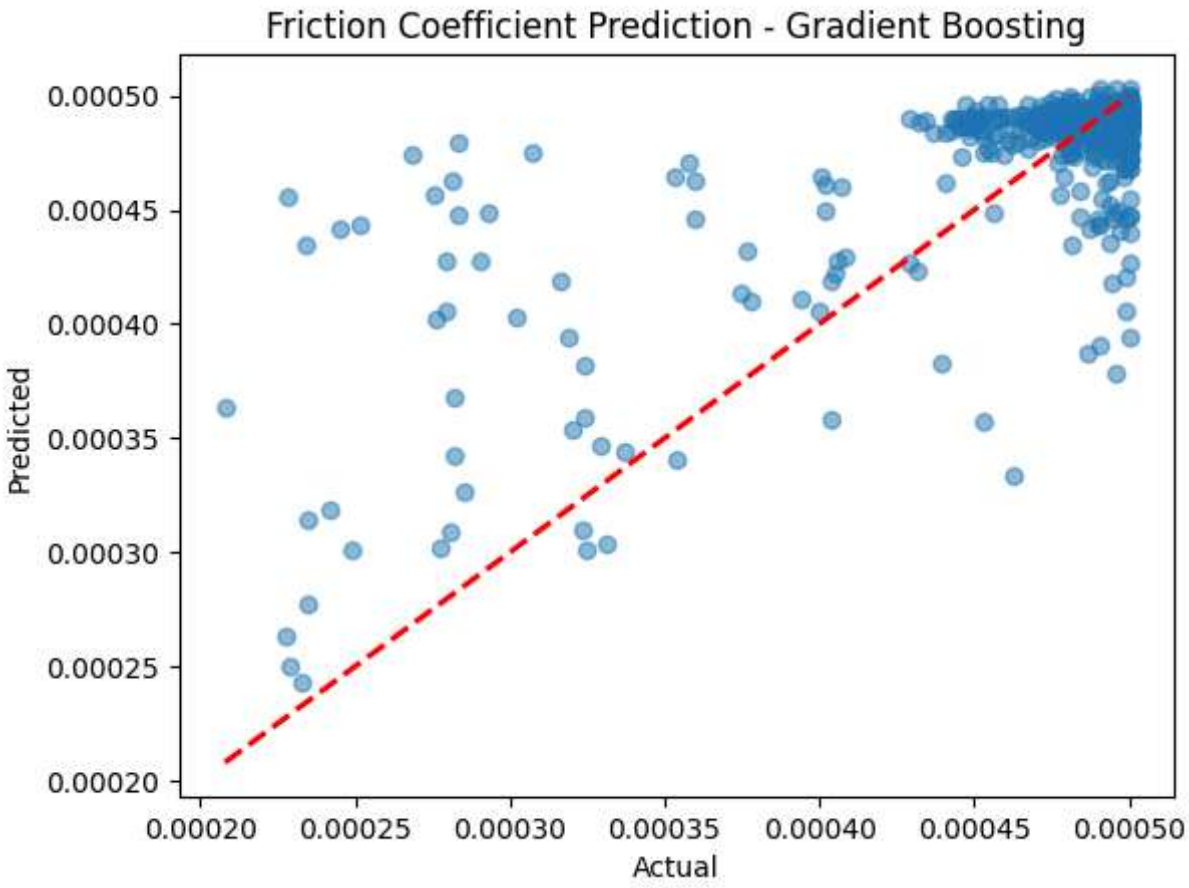
```
=== Gradient Boosting Multi-Output Results ===
Armature Resistance --> RMSE: 0.014537, R²: 0.9190
Armature Inductance  --> RMSE: 0.000122, R²: -0.0412
Torque Constant --> RMSE: 0.003454, R²: 0.2614
Back EMF Constant --> RMSE: 0.002780, R²: 0.9812
Friction Coefficient --> RMSE: 0.000031, R²: 0.5433
Moment of Inertia --> RMSE: 0.000041, R²: 0.5612
```

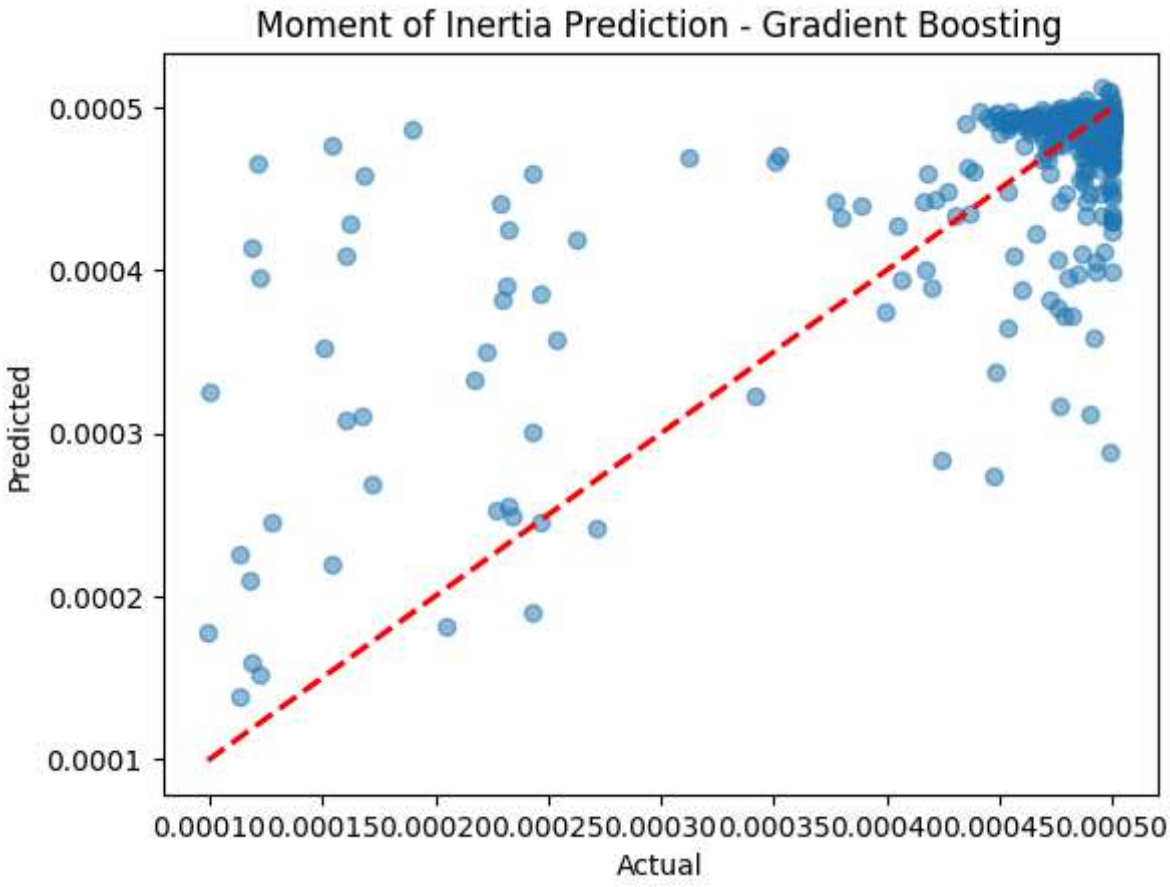

Armature Resistance Prediction - Gradient Boosting
Predicted
Actual
2.000
1.975
1.950
1.925
1.900
1.875
1.850
1.825
1.825
1.850
1.875
1.900
1.925
1.950
1.975
2.000


Armature Inductance Prediction - Gradient Boosting
Predicted
Actual
0.0100
0.0099
0.0098
0.0097
0.0096
0.0095
0.0094
0.0093
0.0093
0.0094
0.0095
0.0096
0.0097
0.0098
0.0099
0.0100


Torque Constant Prediction - Gradient Boosting
Predicted
Actual
0.175
0.180
0.185
0.190
0.195
0.200


Back EMF Constant Prediction - Gradient Boosting
Predicted
Actual
0.13
0.14
0.15
0.16
0.17
0.18
0.19
0.20


Friction Coefficient Prediction - Gradient Boosting
Predicted
Actual
0.00020
0.00025
0.00030
0.00035
0.00040
0.00045
0.00050


Moment of Inertia Prediction - Gradient Boosting
Predicted
Actual
0.0001
0.0002
0.0003
0.0004
0.0005
0.00010 0.00015 0.00020 0.00025 0.00030 0.00035 0.00040 0.00045 0.00050

Support Vector Regressor

In [98]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

input_features = [
 "INPUT_VOLTAGE",
 "TORQUE",
 "SPEED_0",
 "SPEED_1",
 "Acceleration",
 "Speed_Avg",
 "Speed_Diff",
 "Speed_sq",
 "Estimated_Current",
 "Voltage_x_Acceleration",
 "Torque_x_Speed"
]
target_features = [
 "Armature Resistance",
 "Armature Inductance ",
 "Torque Constant",
 "Back EMF Constant",
 "Friction Coefficient",
 "Moment of Inertia"
]

X = df[input_features]
y = df[target_features]

scaler_X = StandardScaler()
X_scaled = scaler_X.fit_transform(X)

scaler_y = StandardScaler()
y_scaled = scaler_y.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_scaled, test_size=0.2, random_state=42
)

base_model = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.001)
model = MultiOutputRegressor(base_model)
model.fit(X_train, y_train)

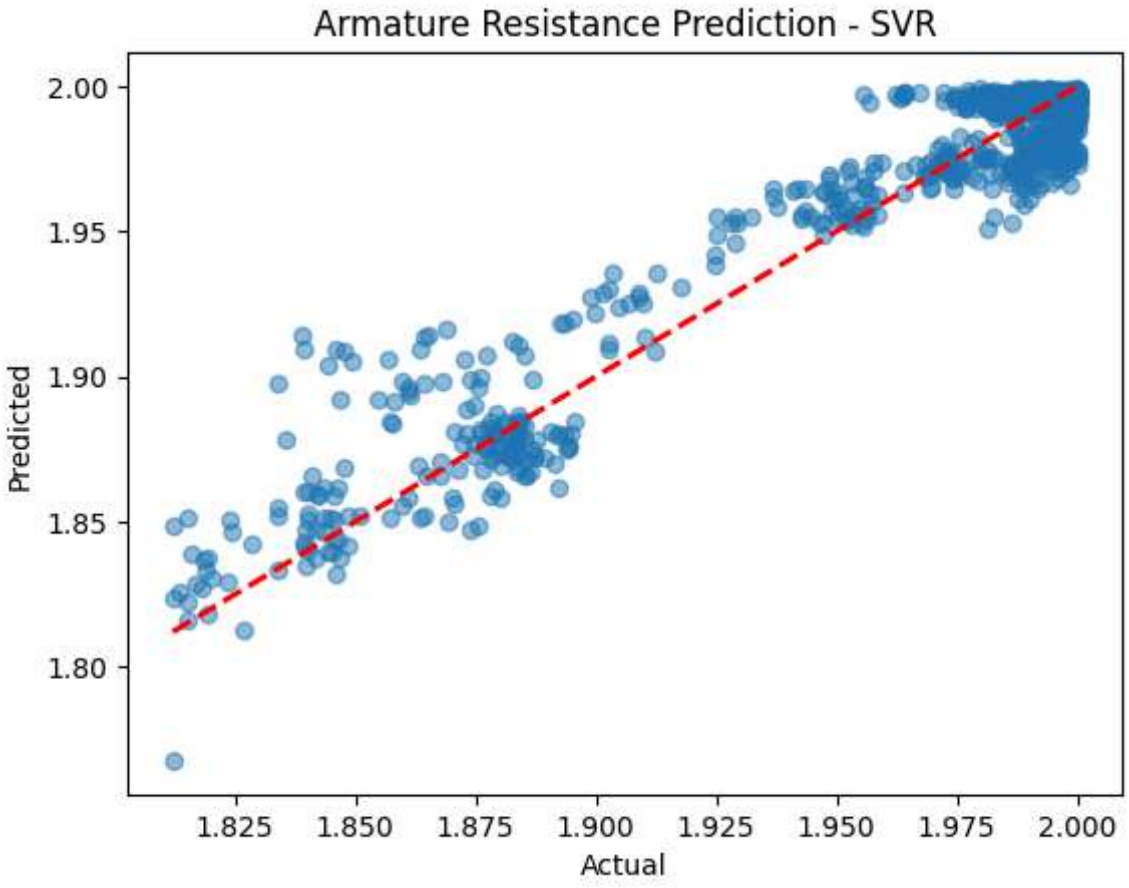
y_pred_scaled = model.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred_scaled)
y_test_orig = scaler_y.inverse_transform(y_test)

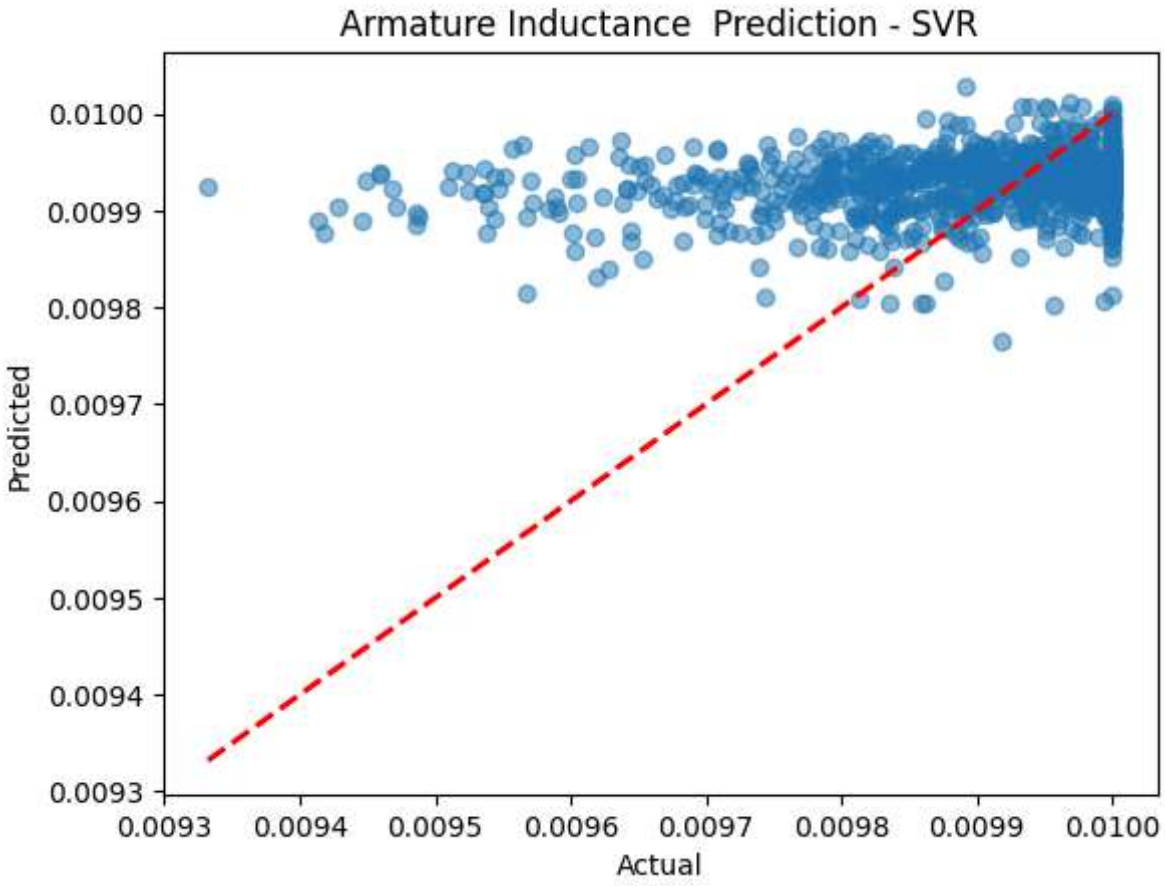
print("=== Multi-Output SVR Results ===")
```

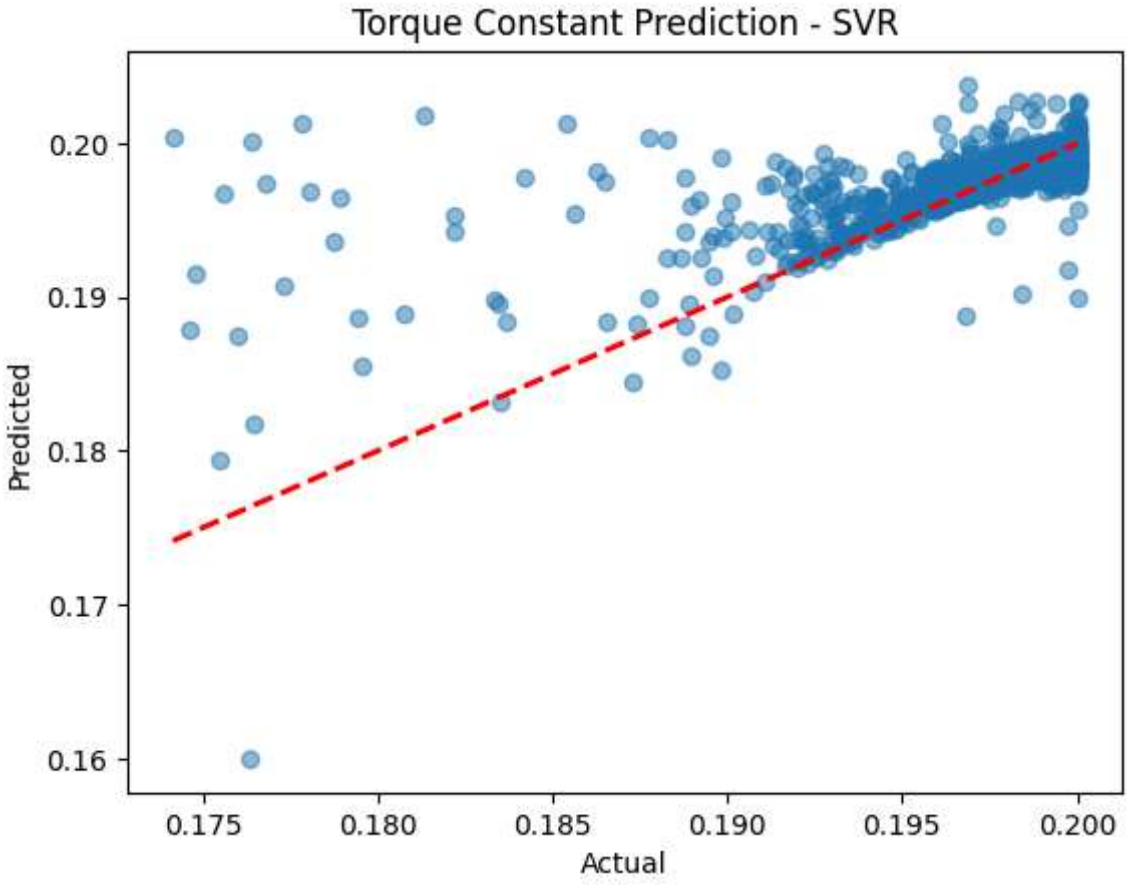
```
for i, col in enumerate(y.columns):
    rmse = np.sqrt(mean_squared_error(y_test_orig[:, i], y_pred[:, i]))
    r2 = r2_score(y_test_orig[:, i], y_pred[:, i])
    print(f"{col} --> RMSE: {rmse:.6f}, R²: {r2:.4f}")

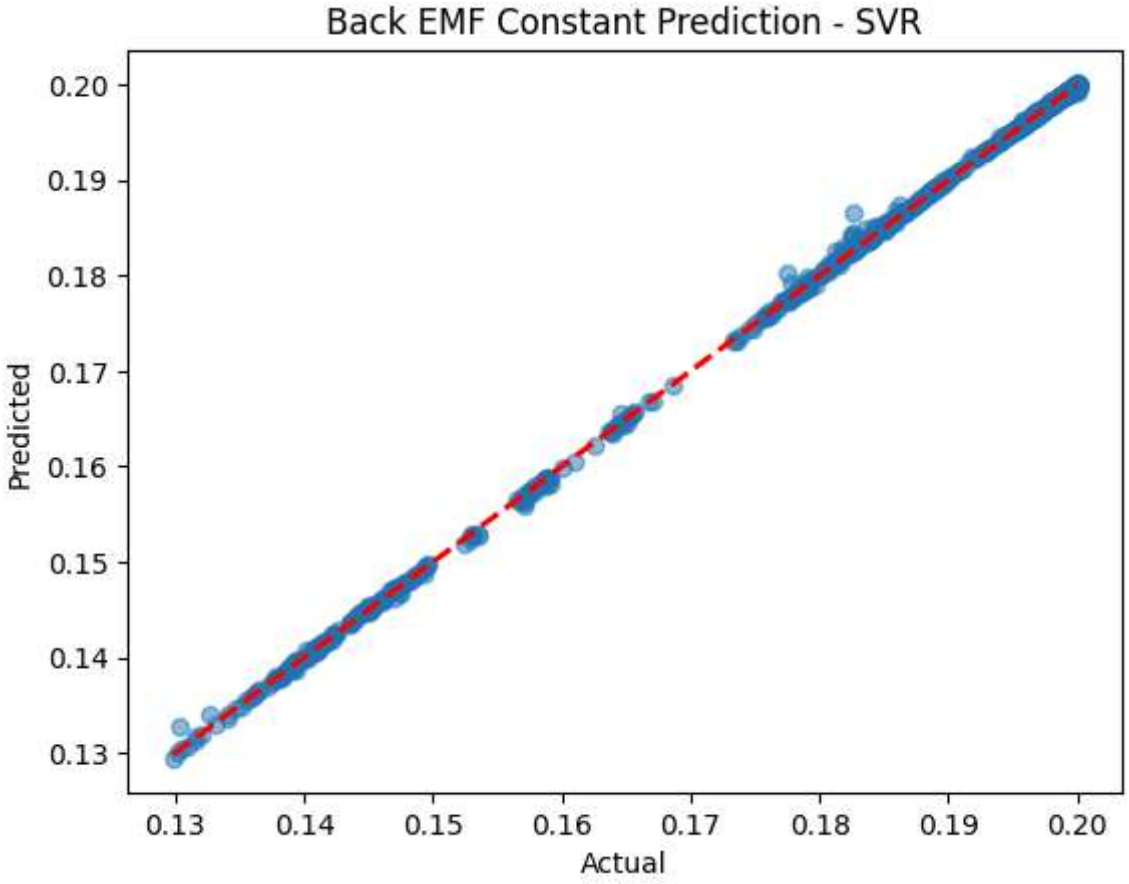
for i, col in enumerate(y.columns):
    plt.figure()
    plt.scatter(y_test_orig[:, i], y_pred[:, i], alpha=0.5)
    plt.plot([y_test_orig[:, i].min(), y_test_orig[:, i].max()],
             [y_test_orig[:, i].min(), y_test_orig[:, i].max()],
             'r--', lw=2)
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.title(f"{col} Prediction - SVR")
    plt.show()
```

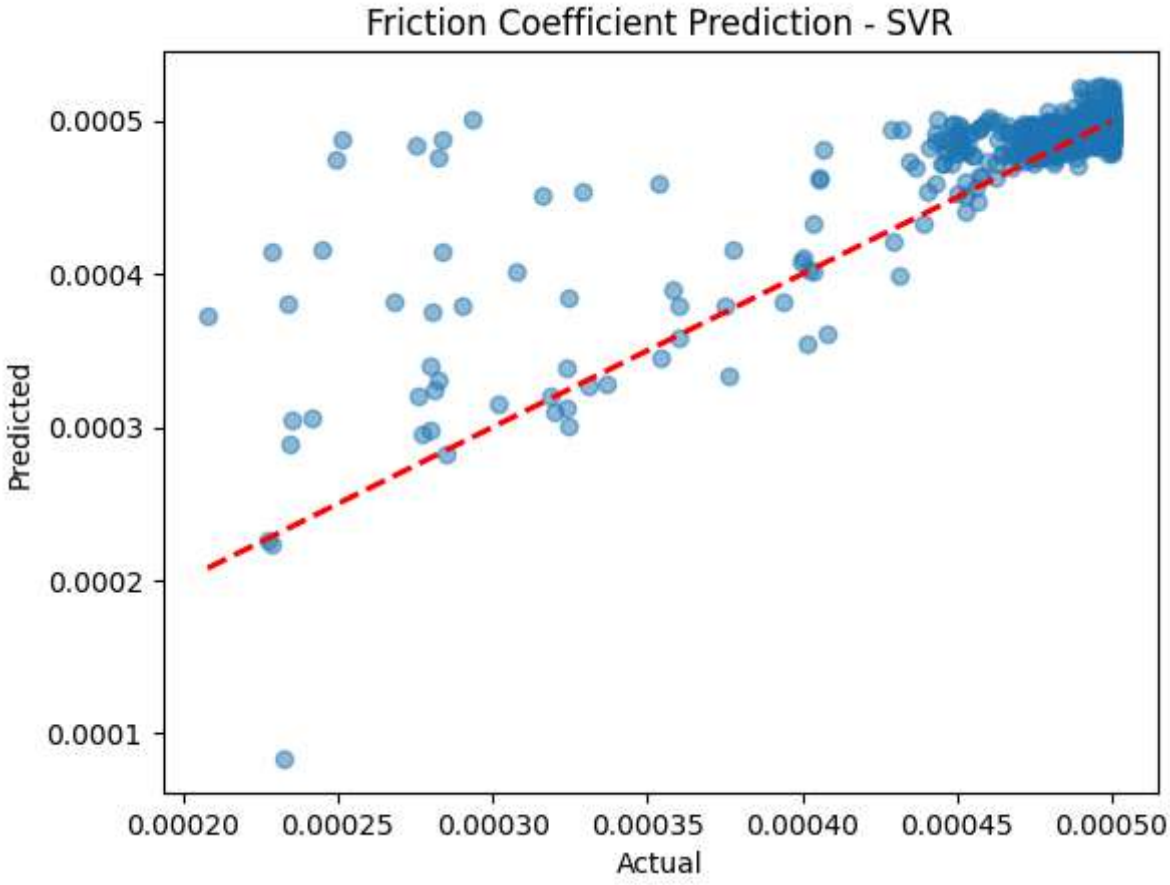
```
=== Multi-Output SVR Results ===
Armature Resistance --> RMSE: 0.013732, R²: 0.9277
Armature Inductance  --> RMSE: 0.000122, R²: -0.0305
Torque Constant --> RMSE: 0.003109, R²: 0.4016
Back EMF Constant --> RMSE: 0.000291, R²: 0.9998
Friction Coefficient --> RMSE: 0.000026, R²: 0.6589
Moment of Inertia --> RMSE: 0.000039, R²: 0.6055
```

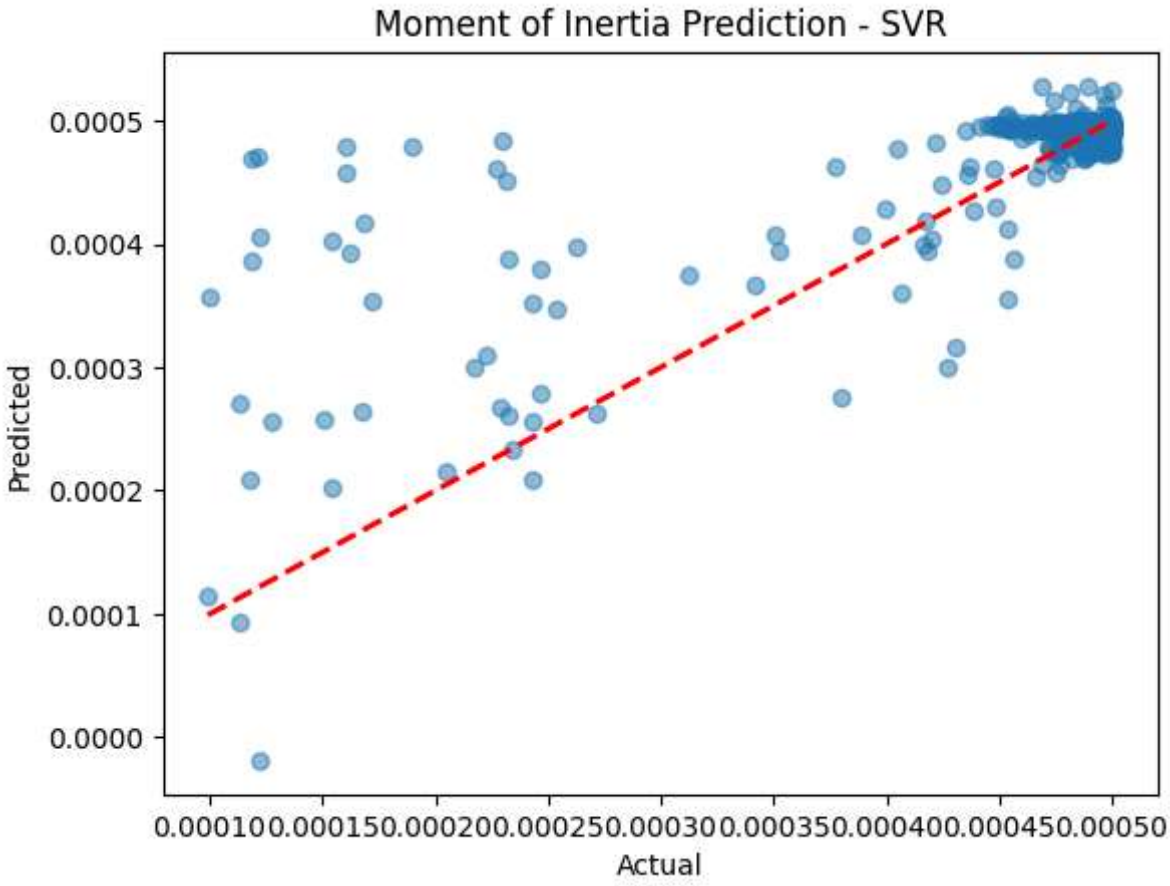



Armature Inductance Prediction - SVR
Predicted
Actual
0.0093
0.0094
0.0095
0.0096
0.0097
0.0098
0.0099
0.0100


Torque Constant Prediction - SVR
Predicted
Actual
0.16
0.17
0.18
0.19
0.20
0.175
0.180
0.185
0.190
0.195
0.200


Back EMF Constant Prediction - SVR
Predicted
Actual
0.13
0.14
0.15
0.16
0.17
0.18
0.19
0.20


Friction Coefficient Prediction - SVR
Predicted
Actual
0.0001
0.0002
0.0003
0.0004
0.0005
0.00020
0.00025
0.00030
0.00035
0.00040
0.00045
0.00050



Making parameters more stronger(by comparison svr turns out to be best)

In [ ]:
```
df['acc_mean'] = df[['SPEED_0','SPEED_1']].diff(axis=1).mean(axis=1)/delta_t
df['acc_max'] = df[['SPEED_0','SPEED_1']].diff(axis=1).max(axis=1)/delta_t
df['acc_min'] = df[['SPEED_0','SPEED_1']].diff(axis=1).min(axis=1)/delta_t
df['acc_abs'] = np.abs(df['Acceleration'])
```

For Armature Inductance

In [185…
```
df['delta_i'] = df['Estimated_Current'].diff().fillna(0)
df['i_dot'] = df['delta_i'] / delta_t  # derivative of current

df['acceleration_diff'] = df['Acceleration'].diff().fillna(0)
df['jerk'] = df['acceleration_diff'] / delta_t

df['V_x_i'] = df['INPUT_VOLTAGE'] * df['Estimated_Current']
df['V_over_i'] = df['INPUT_VOLTAGE'] / (df['Estimated_Current'] + 1e-6)  # avoid di
```

For Torque Constant

In [186…
```
df['T_over_i'] = df['TORQUE'] / (df['Estimated_Current'] + 1e-6)
df['T_x_speed'] = df['TORQUE'] * df['Speed_Avg']

df['T_dot'] = df['TORQUE'].diff().fillna(0) / delta_t
```

For Friction Coefficient

In [187…
```
df['neg_acc'] = df['Acceleration'].apply(lambda x: -x if x < 0 else 0)
df['B_feature'] = df['neg_acc'] / (df['Speed_Avg'] + 1e-6)
```

For Moment Of Inertia

In [188…
```
df['T_over_acc'] = df['TORQUE'] / (df['Acceleration'] + 1e-6)
df['J_feature'] = df['T_over_acc']  # directly proportional to J
```

Check For NaNs

In [189…
```
nan_cols = df.columns[df.isna().any()].tolist()
print("Columns with NaN:", nan_cols)

for c in nan_cols:
    print(c, df[c].isna().sum())
```

```
Columns with NaN: []
```

In [190…
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

input_features = [
 "INPUT_VOLTAGE",
 "TORQUE",
 "SPEED_0",
 "SPEED_1",
 'Acceleration',
 'Speed_Avg',
 'Speed_Diff',
 'Speed_sq',
 'Estimated_Current',
 'T_x_speed',
 'i_dot',
 'jerk',
 'V_x_i',
 'V_over_i',
 'T_over_i',
 'T_dot',
 'neg_acc',
 'T_over_acc',
 'J_feature',
 'acc_mean',
 'acc_max',
 'acc_min',
 'acc_abs'
]
target_features = [
 "Armature Resistance",
 "Armature Inductance ",
```

```
 "Torque Constant",
 "Back EMF Constant",
 "Friction Coefficient",
 "Moment of Inertia"
]

X = df[input_features]
y = df[target_features]

scaler_X = StandardScaler()
X_scaled = scaler_X.fit_transform(X)

scaler_y = StandardScaler()
y_scaled = scaler_y.fit_transform(y)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y_scaled, test_size=0.2, random_state=42
)

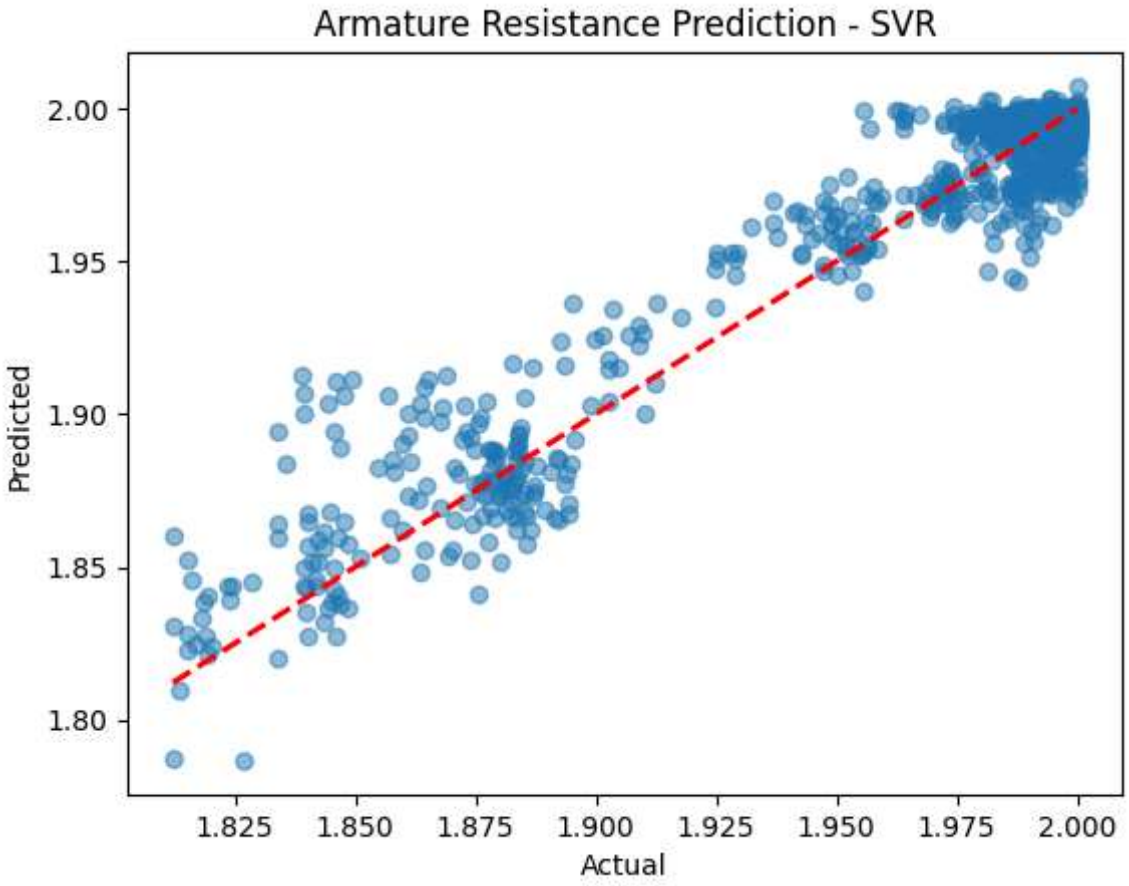
base_model = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.001)
model = MultiOutputRegressor(base_model)
model.fit(X_train, y_train)

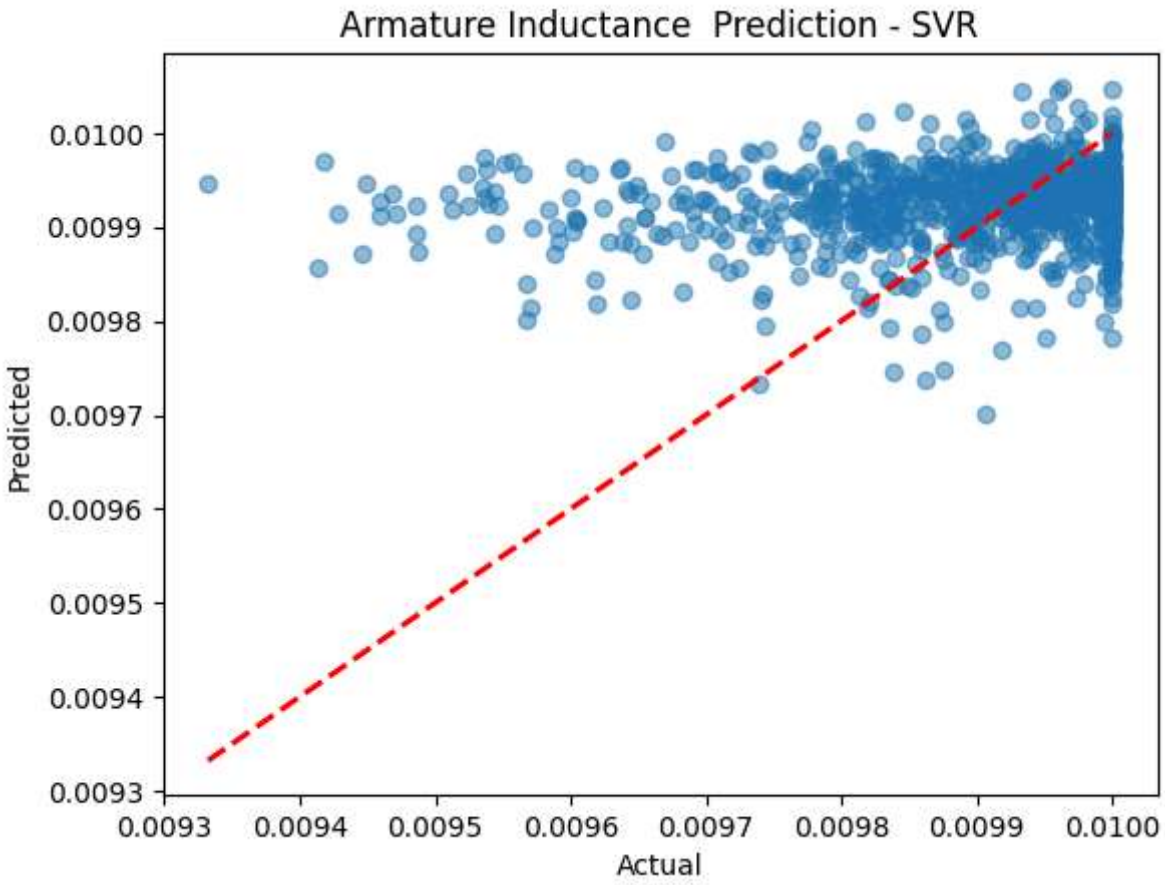
y_pred_scaled = model.predict(X_test)
y_pred = scaler_y.inverse_transform(y_pred_scaled)
y_test_orig = scaler_y.inverse_transform(y_test)

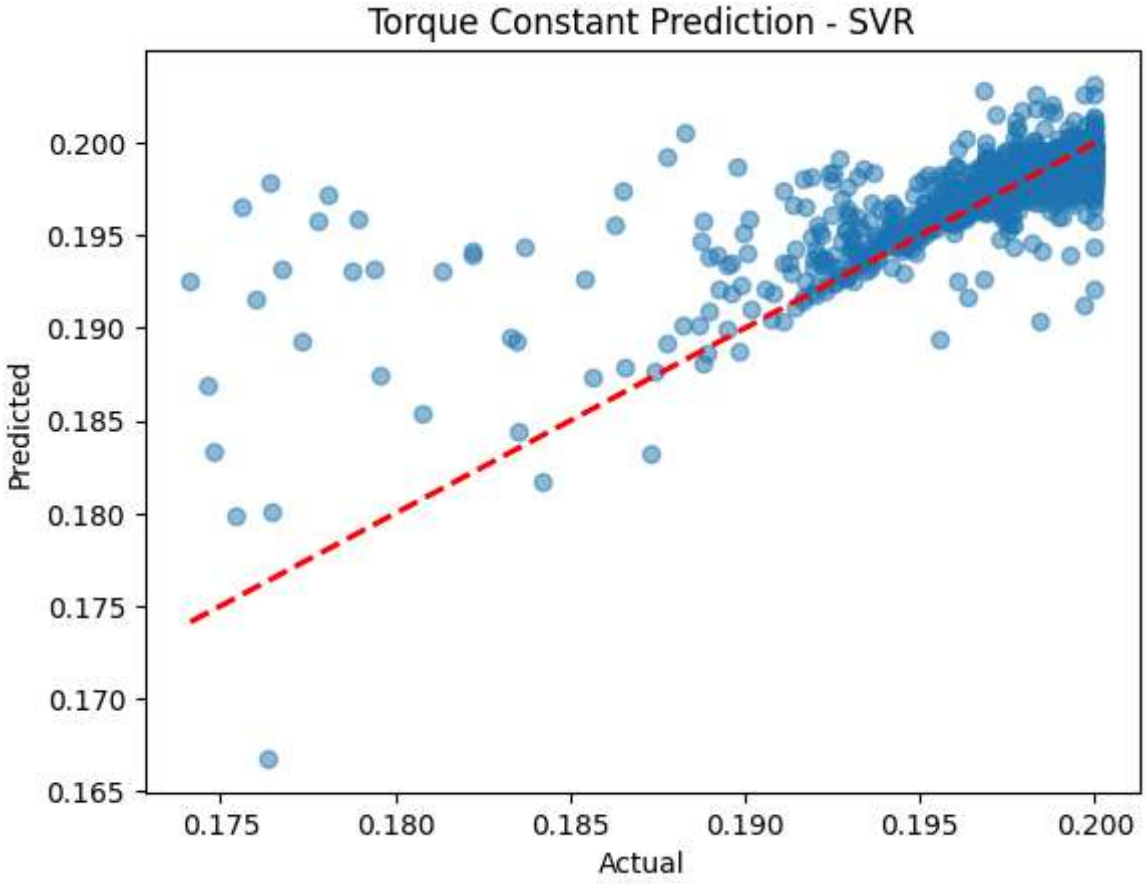
print("=== Multi-Output SVR Results ===")
for i, col in enumerate(y.columns):
    rmse = np.sqrt(mean_squared_error(y_test_orig[:, i], y_pred[:, i]))
    r2 = r2_score(y_test_orig[:, i], y_pred[:, i])
    print(f"{col} --> RMSE: {rmse:.6f}, R²: {r2:.4f}")

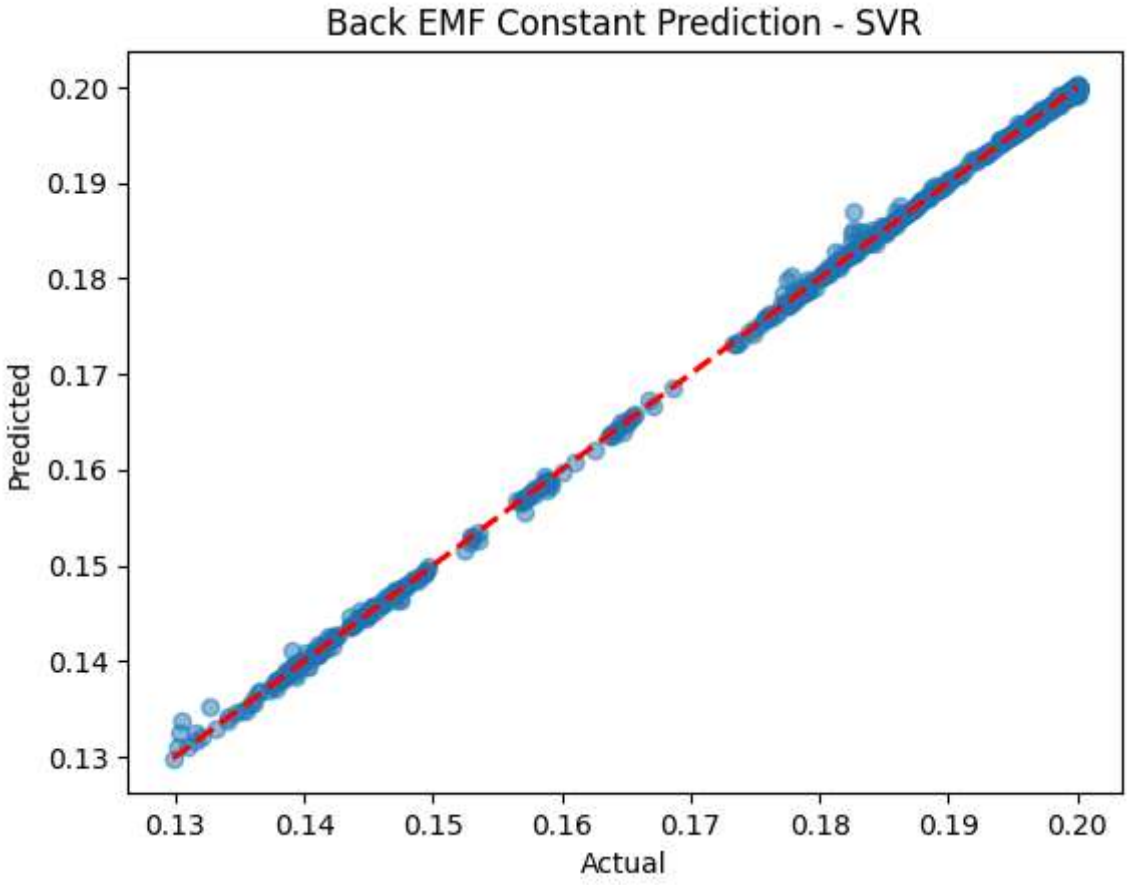
for i, col in enumerate(y.columns):
    plt.figure()
    plt.scatter(y_test_orig[:, i], y_pred[:, i], alpha=0.5)
    plt.plot([y_test_orig[:, i].min(), y_test_orig[:, i].max()],
             [y_test_orig[:, i].min(), y_test_orig[:, i].max()],
             'r--', lw=2)
    plt.xlabel("Actual")
    plt.ylabel("Predicted")
    plt.title(f"{col} Prediction - SVR")
    plt.show()
```

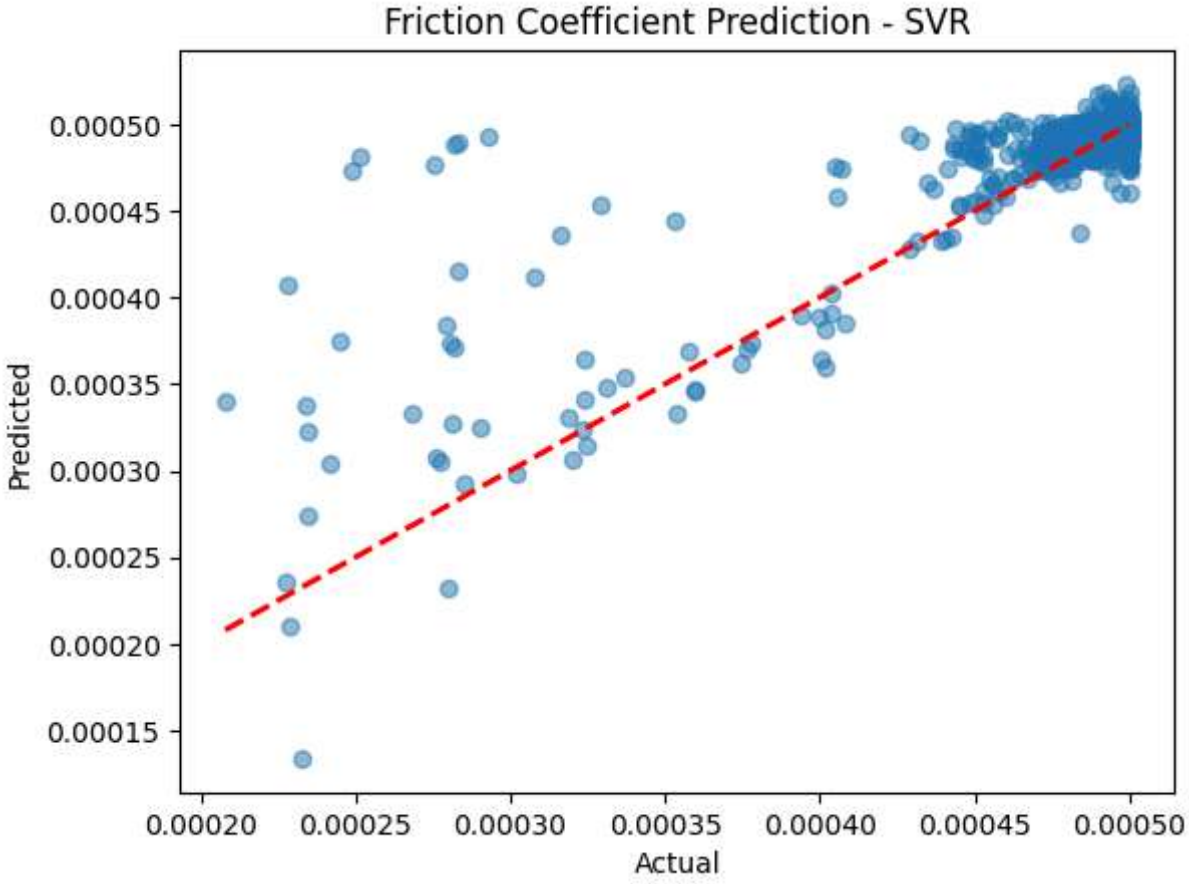
```
=== Multi-Output SVR Results ===
Armature Resistance --> RMSE: 0.014160, R²: 0.9231
Armature Inductance  --> RMSE: 0.000125, R²: -0.0831
Torque Constant --> RMSE: 0.002721, R²: 0.5415
Back EMF Constant --> RMSE: 0.000363, R²: 0.9997
Friction Coefficient --> RMSE: 0.000025, R²: 0.6927
Moment of Inertia --> RMSE: 0.000038, R²: 0.6365
```

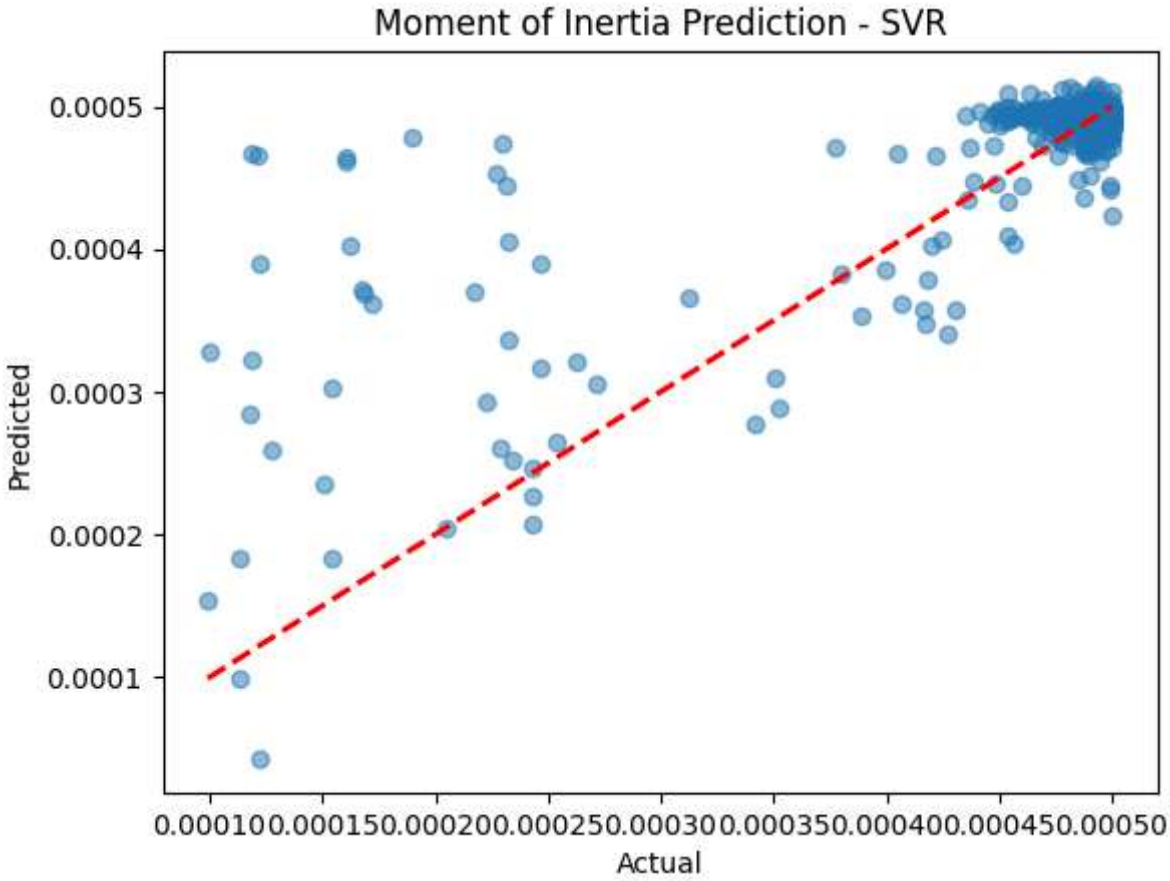

Armature Resistance Prediction - SVR
Predicted
Actual
2.00
1.95
1.90
1.85
1.80
1.825
1.850
1.875
1.900
1.925
1.950
1.975
2.000


Armature Inductance Prediction - SVR
Predicted
Actual
0.0100
0.0099
0.0098
0.0097
0.0096
0.0095
0.0094
0.0093
0.0093
0.0094
0.0095
0.0096
0.0097
0.0098
0.0099
0.0100


Torque Constant Prediction - SVR
Predicted
Actual
0.165
0.170
0.175
0.180
0.185
0.190
0.195
0.200


Back EMF Constant Prediction - SVR
Predicted
Actual
0.13
0.14
0.15
0.16
0.17
0.18
0.19
0.20


Friction Coefficient Prediction - SVR
Predicted
Actual
0.00015
0.00020
0.00025
0.00030
0.00035
0.00040
0.00045
0.00050


Moment of Inertia Prediction - SVR
Predicted
Actual
0.0001
0.0002
0.0003
0.0004
0.0005
0.00010 0.00015 0.00020 0.00025 0.00030 0.00035 0.00040 0.00045 0.00050